Note de séance SQL n°2

Fonctions d'agrégation, GROUP BY et HAVING | 14 octobre 2026

1. L'idée de la séance

Jusqu'ici on affichait des lignes. Un data analyst doit surtout **résumer** les données : compter, faire des moyennes, comparer des catégories. Les fonctions d'agrégation calculent un seul résultat à partir de plusieurs lignes, et **GROUP BY** les applique catégorie par catégorie.

2. Données utilisées (table clients, 8 lignes)

```
INSERT INTO clients VALUES (5, 'Grace', 'Kinshasa', 31);
INSERT INTO clients VALUES (6, 'David', 'Goma', 26);
INSERT INTO clients VALUES (7, 'Sarah', 'Lubumbashi', 38);
INSERT INTO clients VALUES (8, 'Eric', 'Kinshasa', 45);
```

3. Les mots-clés vus aujourd'hui

Mot-clé	Rôle	Exemple
COUNT(*)	Compter les lignes	`SELECT COUNT(*) FROM clients`
AVG(col)	Calculer la moyenne	`SELECT AVG(age) FROM clients`
MIN / MAX	Plus petite / plus grande valeur	`SELECT MIN(age), MAX(age)`
GROUP BY	Regrouper par catégorie	`GROUP BY ville`
HAVING	Filtrer les groupes (après GROUP BY)	`HAVING COUNT(*) >= 3`
AS	Nommer une colonne calculée	`COUNT(*) AS nb_clients`

4. Exercices et corrections

```
-- Nombre total de clients : 8
SELECT COUNT(*) FROM clients;

-- Age moyen : 33.25
SELECT AVG(age) FROM clients;

-- Age moyen par ville : Kinshasa 31.5 / Lubumbashi 36.5 / Goma 33.5
SELECT ville, AVG(age) AS moy_age FROM clients GROUP BY ville;

-- Nombre de clients par ville, du plus grand au plus petit
SELECT ville, COUNT(*) AS nb_clients
FROM clients
GROUP BY ville
ORDER BY nb_clients DESC, ville;
```

5. Mini-défis et corrections

```
-- Villes dont l'âge moyen dépasse 33 : Lubumbashi (36.5), Goma (33.5)
SELECT ville, AVG(age) AS moy_age
FROM clients
GROUP BY ville
```

```
HAVING AVG(age) > 33;

-- Clients de plus de 30 ans par ville : Kinshasa 2 / Lubumbashi 2 / Goma 1
SELECT ville, COUNT(*) AS nb_client
FROM clients
WHERE age > 30
GROUP BY ville;

-- La ville qui a le plus de clients : Kinshasa (4)
SELECT ville, COUNT(*) AS nb_client
FROM clients
GROUP BY ville
ORDER BY nb_client DESC
LIMIT 1;
```

6. À retenir

• **WHERE** filtre les lignes **avant** le regroupement. **HAVING** filtre les groupes **après**.

• Ordre d'écriture : SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT.

• MySQL accepte l'alias dans HAVING (`HAVING moy_age > 33`), mais d'autres systèmes (PostgreSQL, SQL Server) le refusent. La version portable répète le calcul : `HAVING AVG(age) > 33`.

• En cas d'égalité dans un tri, MySQL ne garantit pas l'ordre : ajouter un second critère (`ORDER BY nb_clients DESC, ville`).

• Terminer chaque requête par un point-virgule.

7. Prochaine séance

Les jointures (INNER JOIN, LEFT JOIN) : relier plusieurs tables entre elles, la notion centrale du SQL.